Carbon API — Cidades ESGInteligentes

Testes Automatizados com BDD (Gherkin)
Testes de API, contrato com JSON Schema e execução em CI

Integrante	RM
Gabriel Alves Oliveira	RM562365
Marcos Antão Fernandes Junior	RM563156
Marcos Eduardo Alves de Oliveira Freitas	RM555914
Danilo Ferreira Chaves	RM565765
Hevilyn Romano Garcia	RM566426

FIAP — Análise e Desenvolvimento de Sistemas
Fase: Testes Automatizados
Repositório: https://github.com/GabrielOlliver/carbon-api-bdd-tests

1. Visão geral

A Carbon API é uma API REST em .NET 8 para gestão de emissões, reduções e compensações de carbono, com persistência em Oracle 23ai. Nesta entrega foi construída uma suíte de testes automatizados em **BDD (Behavior Driven Development)**: o comportamento esperado da API é descrito em linguagem natural (Gherkin, em português) e cada frase é ligada a código C# que executa chamadas HTTP reais e valida o resultado.

Requisito do desafio	Como foi atendido
Mínimo de 3 cenários BDD em Gherkin, positivos e negativos	32 cenários (38 execuções, contando as linhas de Exemplos) em 7 funcionalidades, marcados com @positivo e @negativo
Testes de API para todas as APIs	Os 16 endpoints de negócio + 2 endpoints de saúde
Validação de status code	Passo: Então o status da resposta deve ser <código>
Validação do corpo JSON	Passo: E o campo "x" da resposta deve ser "y" (com comparação numérica)
Testes de contrato com JSON Schema	12 schemas draft-07 validados com NJsonSchema em cada resposta
Execução simples e documentada	Um script local (scripts/run-bdd-tests.sh) e um pipeline no GitHub Actions

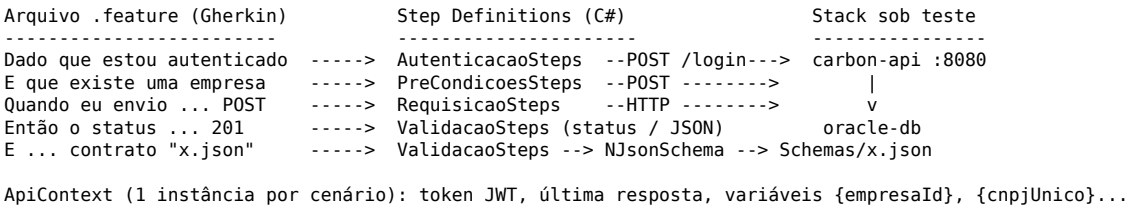
Ferramentas

Ferramenta	Papel
Reqnroll 3.3.4	Framework BDD para .NET. É o sucessor open-source do SpecFlow (sugerido no enunciado, descontinuado em 2024), com a mesma sintaxe Gherkin e os mesmos conceitos (features, step definitions, hooks)
xUnit	Executor dos testes via dotnet test
HttpClient	Chamadas HTTP reais à API
NJsonSchema 11	Validação de contrato (JSON Schema draft-07)
FluentAssertions	Asserções legíveis, com mensagens que mostram o corpo da resposta em caso de falha
Docker Compose	Sobe a API e o Oracle para os testes
GitHub Actions	Execução automática a cada push

Integração com ESG — pilar Governança e compliance

Os cenários com a tag **@governanca** garantem integridade, rastreabilidade e conformidade dos dados ambientais: não há empresas com CNPJ duplicado; o histórico de emissões de uma empresa contém apenas registros dela; emissões com quantidade inválida ou data futura são rejeitadas; agendamentos cancelados não podem ser alterados; e os indicadores publicados são matematicamente coerentes (saldo líquido = total emitido − total compensado). Também há ganho de eficiência computacional: a stack de teste é efêmera e os cenários rodam em paralelo.

2. Arquitetura da suíte de testes



Pasta / arquivo	Responsabilidade
Features/*.feature	Cenários em Gherkin — o QUE a API deve fazer
StepDefinitions/AutenticacaoSteps.cs	Login, ausência de token, token inválido
StepDefinitions/PreCondicoesSteps.cs	Cria empresa, emissão e agendamento pela própria API (passos Dado)
StepDefinitions/RequisicaoSteps.cs	GET / POST / PUT / DELETE genéricos (passos Quando)
StepDefinitions/ValidacaoSteps.cs	Status, campos JSON, listas e contrato (passos Então)
StepDefinitions/GovernancaEsgSteps.cs	Regras de coerência dos indicadores ambientais
Support/ApiContext.cs	Estado do cenário, HttpClient compartilhado, variáveis e leitura de JSON
Support/Hooks.cs	[BeforeTestRun]: verifica /health/ready antes da suíte
Schemas/*.schema.json	Contratos JSON Schema das respostas

Decisões de projeto

• **Sem mocks:** os testes chamam a API real por HTTP, com o Oracle rodando em container. Isso valida a comunicação entre serviços (API ↔ banco) e o comportamento real das regras de negócio.

• **Cenários independentes:** cada cenário cria os próprios dados pelos passos Dado. Nenhum cenário depende da ordem de execução, o que permite rodar em paralelo.

• **Dados únicos por execução:** o CNPJ é gerado aleatoriamente ({cnpjUnico}), então a suíte pode ser repetida sem conflito de duplicidade.

• **Datas dinâmicas:** {ontem}, {amanha}, {daqui90dias} são calculadas na execução — os testes não “envelhecem”.

• **Steps reutilizáveis:** as frases genéricas permitem criar novos cenários apenas escrevendo Gherkin, sem código C# novo.

• **Tags:** @positivo, @negativo, @contrato, @governanca e uma tag por funcionalidade permitem executar subconjuntos (dotnet test --filter "Category=governanca").

3. Cenários de teste em Gherkin

A tabela resume a cobertura; em seguida estão os arquivos .feature completos, exatamente como são executados.

Funcionalidade	Endpoints	Positivos	Negativos
Autenticação	POST /auth/login + proteção JWT	Login válido	Senha incorreta e usuário inexistente (422), sem token (401), token inválido (401)
Empresas	GET lista, GET /{id}, POST	Cadastrar, listar, buscar por id, empresa do seed	CNPJ duplicado (409), id inexistente (404)
Emissões	POST /relatorio-emissao, GET /emissoes-carbono, GET /historico-emissoes/{id}	Registrar, listar, histórico rastreável	Quantidade 0 e negativa, data futura (422), empresa inexistente (404), histórico inexistente (404)
Agendamentos	POST, GET, GET /{id}, PUT /{id}, DELETE /{id}	Criar, listar, buscar, atualizar, cancelar	Fim antes do início e meta 0 (422), alterar cancelado (422), cancelar 2× (422), inexistente (404)
Compensações	POST, GET	Registrar reflorestamento, listar	Quantidade 0 (422), empresa inexistente (404)
Indicadores	GET /indicadores-sustentabilidade, GET /impacto-mitigacao/{id}	Indicadores coerentes, cálculo do impacto	Agendamento inexistente (404)
Saúde	GET /health, GET /health/ready	API no ar, API conectada ao banco	—

3.1 Autenticação — 01_Autenticacao.feature

```
# language: pt
@autenticacao @governanca
Funcionalidade: Autenticação de usuários
  Como gestor de sustentabilidade de uma empresa
  Quero me autenticar na Carbon API
  Para que somente pessoas autorizadas registrem e consultem dados ambientais

  @positivo @contrato
  Cenário: Login com credenciais válidas retorna um token JWT
    Quando eu faço login com o email "admin@esg.com" e a senha "admin123"
    Então o status da resposta deve ser 200
    E a resposta deve respeitar o contrato "login.schema.json"
    E o campo "tipo" da resposta deve ser "Bearer"
    E o campo "role" da resposta deve ser "ADMIN"
    E o campo "token" da resposta deve estar preenchido

  @negativo @contrato
  Esquema do Cenário: Login com credenciais inválidas é recusado
    Quando eu faço login com o email "<email>" e a senha "<senha>"
    Então o status da resposta deve ser 422
    E a resposta deve respeitar o contrato "erro.schema.json"
    E a mensagem de erro deve conter "Email ou senha inválidos"

    Exemplos:
      | situacao            | email             | senha       |
      | senha incorreta     | admin@esg.com     | senhaErrada |
      | usuário inexistente | ninguem@esg.com   | admin123    |

  @negativo
  Cenário: Acesso a um recurso protegido sem token é bloqueado
    Dado que não estou autenticado
    Quando eu envio uma requisição GET para "/api/v1/empresas-sustentaveis"
    Então o status da resposta deve ser 401

  @negativo
  Cenário: Acesso com token inválido é bloqueado
    Dado que estou autenticado com um token inválido
    Quando eu envio uma requisição GET para "/api/v1/indicadores-sustentabilidade"
    Então o status da resposta deve ser 401
```

3.2 Empresas — 02_Empresas.feature

```
# language: pt
@empresas
Funcionalidade: Gestão de empresas sustentáveis
  Como administrador da plataforma ESG
  Quero cadastrar e consultar empresas
  Para manter um cadastro íntegro, sem duplicidade de CNPJ

  Contexto:
    Dado que estou autenticado como administrador

  @positivo @contrato
  Cenário: Cadastrar uma nova empresa
    Quando eu envio uma requisição POST para "/api/v1/empresas-sustentaveis" com o corpo:
      """
      {
        "nome": "Solar Verde Ltda",
        "cnpj": "{cnpjUnico}",
        "setor": "Energia Solar",
        "pais": "Brasil"
      }
      """
    Então o status da resposta deve ser 201
    E a resposta deve respeitar o contrato "empresa.schema.json"
    E o campo "nome" da resposta deve ser "Solar Verde Ltda"
    E o campo "cnpj" da resposta deve ser "{cnpjUnico}"
    E o campo "ativa" da resposta deve ser "true"

  @positivo @contrato
  Cenário: Listar empresas com paginação
    Quando eu envio uma requisição GET para "/api/v1/empresas-sustentaveis?page=1&pageSize=5"
    Então o status da resposta deve ser 200
    E a resposta deve respeitar o contrato "empresa-lista.schema.json"
    E o campo "pageNumber" da resposta deve ser "1"
    E o campo "pageSize" da resposta deve ser "5"
    E a lista "data" da resposta não deve estar vazia

  @positivo @contrato
  Cenário: Consultar uma empresa pelo id
    Dado que existe uma empresa cadastrada
    Quando eu envio uma requisição GET para "/api/v1/empresas-sustentaveis/{empresaId}"
    Então o status da resposta deve ser 200
    E a resposta deve respeitar o contrato "empresa.schema.json"
    E o campo "id" da resposta deve ser "{empresaId}"
    E o campo "cnpj" da resposta deve ser "{empresaCnpj}"

  @negativo @contrato @governanca
  Cenário: Não permitir duas empresas com o mesmo CNPJ
    Dado que existe uma empresa cadastrada
    Quando eu envio uma requisição POST para "/api/v1/empresas-sustentaveis" com o corpo:
      """
      {
        "nome": "Empresa Duplicada",
        "cnpj": "{empresaCnpj}",
        "setor": "Varejo",
        "pais": "Brasil"
      }
      """
    Então o status da resposta deve ser 409
    E a resposta deve respeitar o contrato "erro.schema.json"
    E a mensagem de erro deve conter "já cadastrada"

  @negativo @contrato
  Cenário: Consultar uma empresa inexistente
    Quando eu envio uma requisição GET para "/api/v1/empresas-sustentaveis/999999999"
    Então o status da resposta deve ser 404
    E a resposta deve respeitar o contrato "erro.schema.json"
    E a mensagem de erro deve conter "Empresa não encontrada"

  @positivo @contrato
  Cenário: Consultar a empresa cadastrada pelo seed inicial
    Quando eu envio uma requisição GET para "/api/v1/empresas-sustentaveis/1"
    Então o status da resposta deve ser 200
    E a resposta deve respeitar o contrato "empresa.schema.json"
    E o campo "nome" da resposta deve ser "EcoTech Brasil"
    E o campo "cnpj" da resposta deve ser "12.345.678/0001-99"
```

3.3 Emissões de carbono — 03_EmissoesCarbono.feature

```
# language: pt
@emissoes
Funcionalidade: Registro e rastreabilidade de emissões de carbono
  Como responsável pelo inventário de GEE da empresa
  Quero registrar e consultar as emissões de carbono
  Para manter um histórico rastreável e confiável para auditorias ambientais

  Contexto:
    Dado que estou autenticado como administrador
    E que existe uma empresa cadastrada

  @positivo @contrato
  Cenário: Registrar uma emissão de carbono
    Quando eu envio uma requisição POST para "/api/v1/relatorio-emissao" com o corpo:
      """
      {
        "empresaId": {empresaId},
        "quantidadeTco2e": 150.5,
        "escopo": 1,
        "descricao": "Emissões diretas da frota de veículos",
        "dataEmissao": "{ontem}"
      }
      """
    Então o status da resposta deve ser 201
    E a resposta deve respeitar o contrato "emissao.schema.json"
    E o campo "empresaId" da resposta deve ser "{empresaId}"
    E o campo "quantidadeTco2e" da resposta deve ser "150.5"
    E o campo "escopo" da resposta deve ser "1"

  @positivo @contrato
  Cenário: Listar emissões com paginação
    Dado que existe uma emissão registrada para essa empresa
    Quando eu envio uma requisição GET para "/api/v1/emissoes-carbono?page=1&pageSize=10"
    Então o status da resposta deve ser 200
    E a resposta deve respeitar o contrato "emissao-lista.schema.json"
    E a lista "data" da resposta não deve estar vazia

  @positivo @contrato @governanca
  Cenário: Histórico de emissões contém somente registros da empresa consultada
    Dado que existe uma emissão registrada para essa empresa
    Quando eu envio uma requisição GET para "/api/v1/historico-emissoes/{empresaId}?page=1&pageSize=10"
    Então o status da resposta deve ser 200
    E a resposta deve respeitar o contrato "emissao-lista.schema.json"
    E todos os itens da lista "data" devem ter o campo "empresaId" igual a "{empresaId}"

  @negativo @contrato @governanca
  Esquema do Cenário: Rejeitar registro de emissão inválido
    Quando eu envio uma requisição POST para "/api/v1/relatorio-emissao" com o corpo:
      """
      {
        "empresaId": {empresaId},
        "quantidadeTco2e": <quantidade>,
        "escopo": 1,
        "descricao": "Registro inválido",
        "dataEmissao": "<data>"
      }
      """
    Então o status da resposta deve ser 422
    E a resposta deve respeitar o contrato "erro.schema.json"
    E a mensagem de erro deve conter "<mensagem>"

    Exemplos:
      | regra                    | quantidade | data     | mensagem            |
      | quantidade zero          | 0          | {ontem}  | maior que zero      |
      | quantidade negativa      | -10.5      | {ontem}  | maior que zero      |
      | data de emissão no futuro | 50         | {amanha} | não pode ser futura |

  @negativo @contrato
  Cenário: Registrar emissão para empresa inexistente
    Quando eu envio uma requisição POST para "/api/v1/relatorio-emissao" com o corpo:
      """
      {
        "empresaId": 999999999,
        "quantidadeTco2e": 10,
        "escopo": 2,
        "descricao": "Empresa inexistente",
        "dataEmissao": "{ontem}"
      }
      """
    Então o status da resposta deve ser 404
    E a resposta deve respeitar o contrato "erro.schema.json"
    E a mensagem de erro deve conter "Empresa não encontrada"
```

```
  @negativo @contrato
  Cenário: Consultar histórico de empresa inexistente
    Quando eu envio uma requisição GET para "/api/v1/historico-emissoes/999999999"
    Então o status da resposta deve ser 404
    E a resposta deve respeitar o contrato "erro.schema.json"
```

3.4 Agendamentos de redução — 04_AgendamentosReducao.feature

```
# language: pt
@agendamentos
Funcionalidade: Agendamentos de redução de emissões
  Como gestor ambiental
  Quero planejar, acompanhar e cancelar iniciativas de redução de carbono
  Para cumprir as metas de descarbonização da empresa

  Contexto:
    Dado que estou autenticado como administrador
    E que existe uma empresa cadastrada

  @positivo @contrato
  Cenário: Criar um agendamento de redução
    Quando eu envio uma requisição POST para "/api/v1/agendamento-reducao-carbono" com o corpo:
      """
      {
        "empresaId": {empresaId},
        "titulo": "Substituição de frota diesel por elétrica",
        "descricao": "Substituir 30% da frota até o fim do ano",
        "metaReducaoTco2e": 75.25,
        "dataInicio": "{hoje}",
        "dataFimPrevista": "{daqui90dias}"
      }
      """
    Então o status da resposta deve ser 201
    E a resposta deve respeitar o contrato "agendamento.schema.json"
    E o campo "status" da resposta deve ser "1"
    E o campo "metaReducaoTco2e" da resposta deve ser "75.25"

  @positivo @contrato
  Cenário: Listar agendamentos com paginação
    Dado que existe um agendamento de redução cadastrado para essa empresa
    Quando eu envio uma requisição GET para "/api/v1/agendamento-reducao-carbono?page=1&pageSize=10"
    Então o status da resposta deve ser 200
    E a resposta deve respeitar o contrato "agendamento-lista.schema.json"
    E a lista "data" da resposta não deve estar vazia

  @positivo @contrato
  Cenário: Consultar um agendamento pelo id
    Dado que existe um agendamento de redução cadastrado para essa empresa
    Quando eu envio uma requisição GET para "/api/v1/agendamento-reducao-carbono/{agendamentoId}"
    Então o status da resposta deve ser 200
    E a resposta deve respeitar o contrato "agendamento.schema.json"
    E o campo "id" da resposta deve ser "{agendamentoId}"
    E o campo "empresaId" da resposta deve ser "{empresaId}"

  @positivo @contrato
  Cenário: Atualizar o andamento de um agendamento
    Dado que existe um agendamento de redução cadastrado para essa empresa
    Quando eu envio uma requisição PUT para "/api/v1/agendamento-reducao-carbono/{agendamentoId}" com o corpo:
      """
      {
        "titulo": "Substituição de frota - revisado",
        "descricao": "Meta ampliada para 50% da frota",
        "metaReducaoTco2e": 120,
        "status": 2,
        "dataFimPrevista": "{daqui90dias}"
      }
      """
    Então o status da resposta deve ser 200
    E a resposta deve respeitar o contrato "agendamento.schema.json"
    E o campo "status" da resposta deve ser "2"
    E o campo "titulo" da resposta deve ser "Substituição de frota - revisado"

  @positivo
  Cenário: Cancelar um agendamento
    Dado que existe um agendamento de redução cadastrado para essa empresa
    Quando eu envio uma requisição DELETE para "/api/v1/agendamento-reducao-carbono/{agendamentoId}"
    Então o status da resposta deve ser 204
    Quando eu envio uma requisição GET para "/api/v1/agendamento-reducao-carbono/{agendamentoId}"
    Então o campo "status" da resposta deve ser "4"

  @negativo @contrato
  Esquema do Cenário: Rejeitar agendamento com dados inválidos
    Quando eu envio uma requisição POST para "/api/v1/agendamento-reducao-carbono" com o corpo:
      """
```

```
      {
        "empresaId": {empresaId},
        "titulo": "Agendamento inválido",
        "metaReducaoTco2e": <meta>,
        "dataInicio": "<inicio>",
        "dataFimPrevista": "<fim>"
      }
      """
    Então o status da resposta deve ser 422
    E a resposta deve respeitar o contrato "erro.schema.json"
    E a mensagem de erro deve conter "<mensagem>"

    Exemplos:
      | regra                        | meta | inicio        | fim           | mensagem       |
      | data fim anterior ao início | 50   | {daqui90dias} | {daqui30dias} | posterior      |
      | meta de redução zerada      | 0    | {hoje}        | {daqui90dias} | maior que zero |

  @negativo @contrato @governanca
  Cenário: Não permitir alterar um agendamento cancelado
    Dado que existe um agendamento de redução cadastrado para essa empresa
    E que esse agendamento foi cancelado
    Quando eu envio uma requisição PUT para "/api/v1/agendamento-reducao-carbono/{agendamentoId}" com o corpo:
      """
      {
        "titulo": "Tentativa de reativar",
        "metaReducaoTco2e": 10,
        "status": 2,
        "dataFimPrevista": "{daqui90dias}"
      }
      """
    Então o status da resposta deve ser 422
    E a resposta deve respeitar o contrato "erro.schema.json"
    E a mensagem de erro deve conter "cancelado"

  @negativo @contrato
  Cenário: Não permitir cancelar duas vezes o mesmo agendamento
    Dado que existe um agendamento de redução cadastrado para essa empresa
    E que esse agendamento foi cancelado
    Quando eu envio uma requisição DELETE para "/api/v1/agendamento-reducao-carbono/{agendamentoId}"
    Então o status da resposta deve ser 422
    E a mensagem de erro deve conter "já está cancelado"

  @negativo @contrato
  Cenário: Consultar um agendamento inexistente
    Quando eu envio uma requisição GET para "/api/v1/agendamento-reducao-carbono/999999999"
    Então o status da resposta deve ser 404
    E a resposta deve respeitar o contrato "erro.schema.json"
```

3.5 Compensações de carbono — 05_CompensacoesCarbono.feature

```
# language: pt
@compensacoes
Funcionalidade: Compensação de carbono
  Como gestor ambiental
  Quero registrar ações de compensação, como reflorestamento e créditos de carbono
  Para abater as emissões que não puderam ser evitadas

  Contexto:
    Dado que estou autenticado como administrador
    E que existe uma empresa cadastrada

  @positivo @contrato
  Cenário: Registrar uma compensação por reflorestamento
    Quando eu envio uma requisição POST para "/api/v1/compensacao-carbono" com o corpo:
      """
      {
        "empresaId": {empresaId},
        "tipo": 2,
        "quantidadeTco2e": 30,
        "descricao": "Plantio de 1000 árvores nativas no Cerrado",
        "certificadoUrl": "https://certificados.esg.com.br/ref-2026-001",
        "dataCompensacao": "{ontem}"
      }
      """
    Então o status da resposta deve ser 201
    E a resposta deve respeitar o contrato "compensacao.schema.json"
    E o campo "tipo" da resposta deve ser "2"
    E o campo "empresaId" da resposta deve ser "{empresaId}"

  @positivo @contrato
  Cenário: Listar compensações com paginação
    Quando eu envio uma requisição GET para "/api/v1/compensacao-carbono?page=1&pageSize=10"
    Então o status da resposta deve ser 200
    E a resposta deve respeitar o contrato "compensacao-lista.schema.json"
```

```
  @negativo @contrato
  Cenário: Rejeitar compensação com quantidade zerada
    Quando eu envio uma requisição POST para "/api/v1/compensacao-carbono" com o corpo:
      """
      {
        "empresaId": {empresaId},
        "tipo": 1,
        "quantidadeTco2e": 0,
        "dataCompensacao": "{ontem}"
      }
      """
    Então o status da resposta deve ser 422
    E a resposta deve respeitar o contrato "erro.schema.json"
    E a mensagem de erro deve conter "maior que zero"

  @negativo @contrato
  Cenário: Rejeitar compensação para empresa inexistente
    Quando eu envio uma requisição POST para "/api/v1/compensacao-carbono" com o corpo:
      """
      {
        "empresaId": 999999999,
        "tipo": 3,
        "quantidadeTco2e": 5,
        "dataCompensacao": "{ontem}"
      }
      """
    Então o status da resposta deve ser 404
    E a resposta deve respeitar o contrato "erro.schema.json"
```

3.6 Indicadores de sustentabilidade — 06_Indicadores.feature

```
# language: pt
@indicadores @governanca
Funcionalidade: Indicadores de sustentabilidade
  Como auditor ESG
  Quero consultar indicadores consolidados e o impacto das ações de redução
  Para verificar se os números publicados são íntegros e coerentes

  Contexto:
    Dado que estou autenticado como administrador

  @positivo @contrato
  Cenário: Indicadores consolidados são consistentes
    Quando eu envio uma requisição GET para "/api/v1/indicadores-sustentabilidade"
    Então o status da resposta deve ser 200
    E a resposta deve respeitar o contrato "indicadores.schema.json"
    E o campo "totalEmpresas" da resposta deve ser maior ou igual a 1
    E o saldo líquido deve ser igual ao total emitido menos o total compensado

  @positivo @contrato
  Cenário: Calcular o impacto de mitigação de um agendamento
    Dado que existe uma empresa cadastrada
    E que existe uma emissão registrada para essa empresa
    E que existe um agendamento de redução cadastrado para essa empresa
    Quando eu envio uma requisição GET para "/api/v1/impacto-mitigacao/{agendamentoId}"
    Então o status da resposta deve ser 200
    E a resposta deve respeitar o contrato "impacto-mitigacao.schema.json"
    E o campo "agendamentoId" da resposta deve ser "{agendamentoId}"
    E o percentual de impacto deve corresponder à meta dividida pelo total emitido

  @negativo @contrato
  Cenário: Impacto de mitigação para agendamento inexistente
    Quando eu envio uma requisição GET para "/api/v1/impacto-mitigacao/999999999"
    Então o status da resposta deve ser 404
    E a resposta deve respeitar o contrato "erro.schema.json"
```

3.7 Saúde da aplicação — 07_SaudeDaAplicacao.feature

```
# language: pt
@saude
Funcionalidade: Saúde da aplicação
  Como equipe de operações
  Quero verificar se a API e o banco de dados estão disponíveis
  Para detectar falhas antes que afetem os usuários

  @positivo
  Cenário: A API está no ar
    Quando eu envio uma requisição GET para "/health"
    Então o status da resposta deve ser 200
    E o corpo da resposta deve ser "Healthy"

  @positivo
  Cenário: A API está conectada ao banco de dados
    Quando eu envio uma requisição GET para "/health/ready"
    Então o status da resposta deve ser 200
    E o corpo da resposta deve ser "Healthy"
```

4. Testes de contrato com JSON Schema

Cada resposta é validada contra um schema que define os campos obrigatórios (required), os tipos, os valores permitidos (enum), formatos (pattern) e proíbe campos não declarados (additionalProperties: false). Se a API adicionar, remover ou mudar o tipo de um campo, o teste de contrato falha — protegendo os consumidores da API (front-end, integrações, relatórios ESG).

Schema	Valida a resposta de
login.schema.json	POST /auth/login
erro.schema.json	Todas as respostas de erro (404, 409, 422) — formato padronizado do ExceptionMiddleware
empresa / empresa-lista	Empresa individual e lista paginada
emissao / emissao-lista	Emissão individual, lista e histórico por empresa
agendamento / agendamento-lista	Agendamento individual e lista paginada
compensacao / compensacao-lista	Compensação individual e lista paginada
indicadores.schema.json	GET /indicadores-sustentabilidade
impacto-mitigacao.schema.json	GET /impacto-mitigacao/{id}

Exemplo: empresa.schema.json

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "Empresa",
  "type": "object",
  "additionalProperties": false,
  "required": [
    "id",
    "nome",
    "cnpj",
    "setor",
    "pais",
    "ativa",
    "dataCadastro"
  ],
  "properties": {
    "id": {
      "type": "integer",
      "minimum": 1
    },
    "nome": {
      "type": "string",
      "minLength": 1
    },
    "cnpj": {
      "type": "string",
      "pattern": "^\\d{2}\\.\\d{3}\\.\\d{3}/\\d{4}-\\d{2}$"
    },
    "setor": {
      "type": "string"
    },
    "pais": {
      "type": "string"
    },
    "ativa": {
      "type": "boolean"
    },
    "dataCadastro": {
      "type": "string",
      "pattern": "^\\d{4}-\\d{2}-\\d{2}T\\d{2}:\\d{2}:\\d{2}"
    }
  }
}
```

Implementação do passo de contrato (ValidacaoSteps.cs)

```
[Then("a resposta deve respeitar o contrato {string}")]
public async Task EntaoARespostaDeveRespeitarOContrato(string arquivo)
{
    var caminho = Path.Combine(AppContext.BaseDirectory, "Schemas", arquivo);
    var schema = await JsonSchema.FromFileAsync(caminho);
    var erros = schema.Validate(_api.ResponseBody);
    erros.Should().BeEmpty("a resposta deve seguir o contrato {0}. Violacoes: {1}", ...);
}
```

5. Configurações

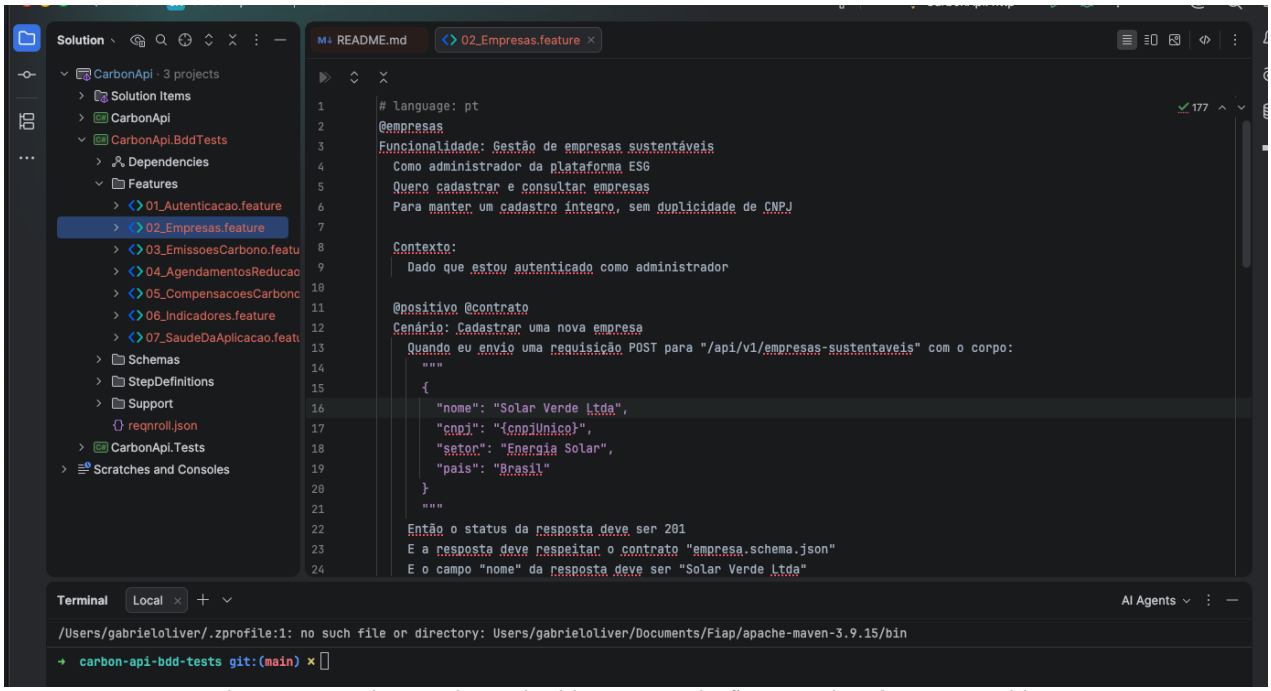


Figura 1 — Projeto CarbonApi.BddTests na solução e arquivo .feature no Rider.

reqnroll.json — Gherkin em português e relatório HTML

```
{
  "$schema": "https://schemas.reqnroll.net/reqnroll-config-latest.json",
  "language": { "feature": "pt" },
  "formatters": {
    "html": { "outputFilePath": "reqnroll_report.html" }
  }
}
```

Pacotes do projeto de testes (CarbonApi.BddTests.csproj)

```
    <!-- Reqnroll: sucessor open-source do SpecFlow (BDD com Gherkin para .NET) -->
    <PackageReference Include="Reqnroll.xUnit" Version="3.3.4" />
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.11.1" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2">
    </PackageReference>
    <!-- Assercoes legiveis -->
    <PackageReference Include="FluentAssertions" Version="6.12.0" />
    <!-- Validacao de contrato com JSON Schema -->
    <PackageReference Include="NJsonSchema" Version="11.0.2" />
```

Execução local — scripts/run-bdd-tests.sh

```
#!/usr/bin/env bash
# Executa a suite BDD localmente:
#   1. sobe API + Oracle com docker compose
#   2. espera a API ficar pronta
#   3. roda os cenarios Gherkin (Reqnroll)
#   4. mostra onde estao os relatorios
set -euo pipefail
cd "$(dirname "$0")/.."

if [ ! -f .env ]; then
  echo "Arquivo .env nao encontrado. Crie com: cp .env.example .env (e ajuste as senhas)"; exit 1
fi

echo ">> Subindo a stack (API + Oracle)..."
docker compose up -d --build

echo ">> Aguardando a API em http://localhost:8080/health/ready ..."
for i in $(seq 1 60); do
  if curl -fsS http://localhost:8080/health/ready >/dev/null 2>&1; then echo "   API pronta."; break; fi
  [ "$i" -eq 60 ] && { echo "API nao ficou pronta. Veja: docker compose logs carbon-api"; exit 1; }
  sleep 5
done
```

```
echo ">> Executando os cenarios BDD..."
dotnet test tests/CarbonApi.BddTests/CarbonApi.BddTests.csproj \
  --logger "console;verbosity=normal" \
  --logger "trx;LogFileName=bdd-tests.trx" \
  --results-directory TestResults

echo ""
echo ">> Relatorio HTML: tests/CarbonApi.BddTests/bin/Debug/net8.0/reqnroll_report.html"
echo ">> Relatorio TRX : TestResults/bdd-tests.trx"
```

Pipeline de CI — .github/workflows/ci-cd.yml

```
name: CI - Testes Automatizados (Carbon API)

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
  workflow_dispatch:

permissions:
  contents: read

jobs:
  # ================================================================
  # 1) Build da solucao + testes unitarios (xUnit + Moq, sem banco)
  # ================================================================
  build-test:
    name: Build e Testes Unitarios
    runs-on: ubuntu-latest
    steps:
      - name: Checkout do codigo
        uses: actions/checkout@v4

      - name: Instalar .NET 8
        uses: actions/setup-dotnet@v4
        with:
          dotnet-version: '8.0.x'

      - name: Restore e build (Release)
        run: |
          dotnet restore CarbonApi.sln
          dotnet build CarbonApi.sln -c Release --no-restore

      - name: Testes unitarios
        run: >
          dotnet test tests/CarbonApi.Tests/CarbonApi.Tests.csproj -c Release --no-build
          --logger "trx;LogFileName=unit-tests.trx"
          --results-directory TestResults

      - name: Publicar resultado dos testes unitarios
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: testes-unitarios
          path: TestResults

  # ================================================================
  # 2) Testes BDD de API (Reqnroll + Gherkin) contra a stack real
  #    API + Oracle sobem com docker compose dentro do runner
  # ================================================================
  bdd-tests:
    name: Testes BDD de API (Gherkin + JSON Schema)
    runs-on: ubuntu-latest
    needs: build-test
    env:
      API_BASE_URL: http://localhost:8080
    steps:
      - name: Checkout do codigo
        uses: actions/checkout@v4

      - name: Gerar .env efemero (senhas aleatorias, so para este run)
        run: |
          cat > .env <<ENV
          ORACLE_PASSWORD=Ora$(openssl rand -hex 8)
          APP_USER=carbon_app
          APP_USER_PASSWORD=App$(openssl rand -hex 8)
          JWT_SECRET=$(openssl rand -hex 32)
          ASPNETCORE_ENVIRONMENT=Development
          API_PORT=8080
          DB_PORT=1521
          ENV

      - name: Subir a stack (API + Oracle)
```

```
    run: docker compose up -d --build

  - name: Aguardar a API ficar pronta (/health/ready)
    run: |
      for i in $(seq 1 60); do
        if curl -fsS http://localhost:8080/health/ready; then echo ""; echo "API pronta"; exit 0; fi
        echo "Aguardando API + Oracle... ($i/60)"; sleep 10
      done
      docker compose logs --tail=100
      exit 1

  - name: Instalar .NET 8
    uses: actions/setup-dotnet@v4
    with:
      dotnet-version: '8.0.x'

  - name: Executar cenarios BDD
    run: >
      dotnet test tests/CarbonApi.BddTests/CarbonApi.BddTests.csproj -c Release
      --logger "trx;LogFileName=bdd-tests.trx"
      --logger "console;verbosity=normal"
      --results-directory TestResults

  - name: Publicar relatorios (TRX + HTML do Reqnroll)
    if: always()
    uses: actions/upload-artifact@v4
    with:
      name: relatorios-bdd
      path: |
        TestResults
        tests/CarbonApi.BddTests/bin/Release/net8.0/reqnroll_report.html

  - name: Logs da API (somente em caso de falha)
    if: failure()
    run: docker compose logs --tail=200 carbon-api

  - name: Derrubar a stack
    if: always()
    run: docker compose down -v
```

6. Evidências de execução

Execução local

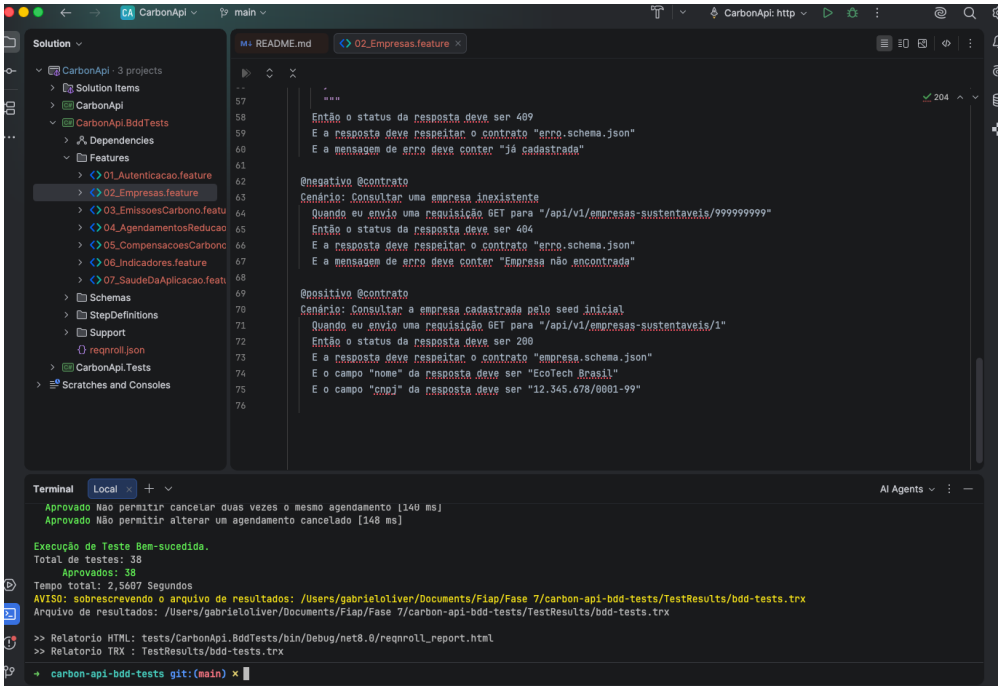


Figura 2 — Execução local pelo script: 38 testes aprovados.

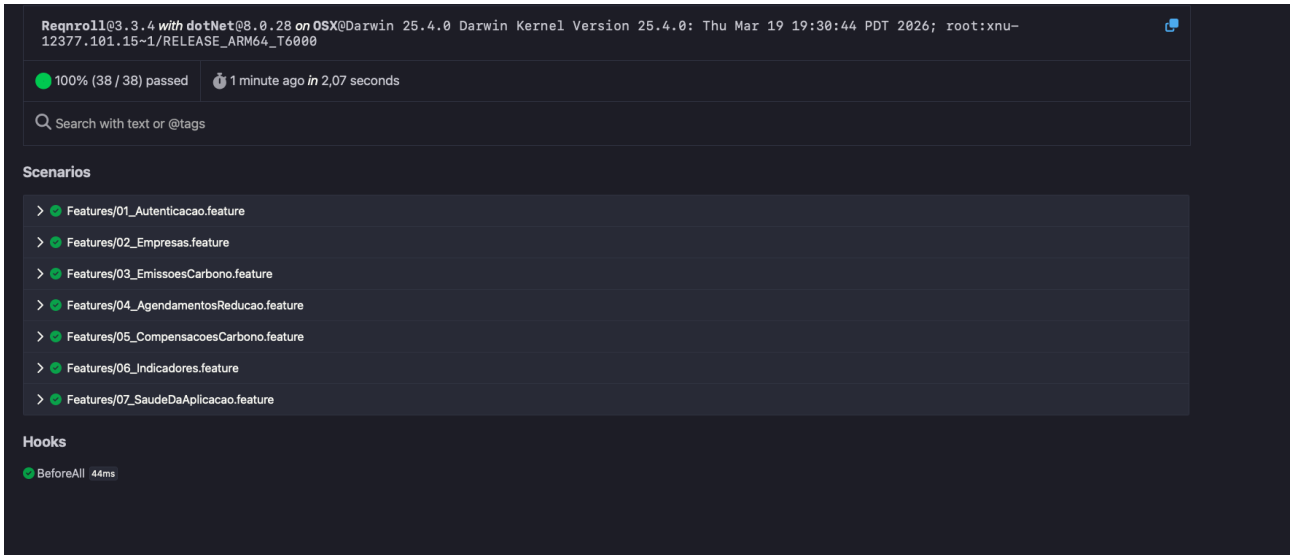


Figura 3 — Relatório HTML gerado pelo Reqnroll: 100% (38/38) aprovados nas 7 funcionalidades.

Execução no pipeline de CI (GitHub Actions)

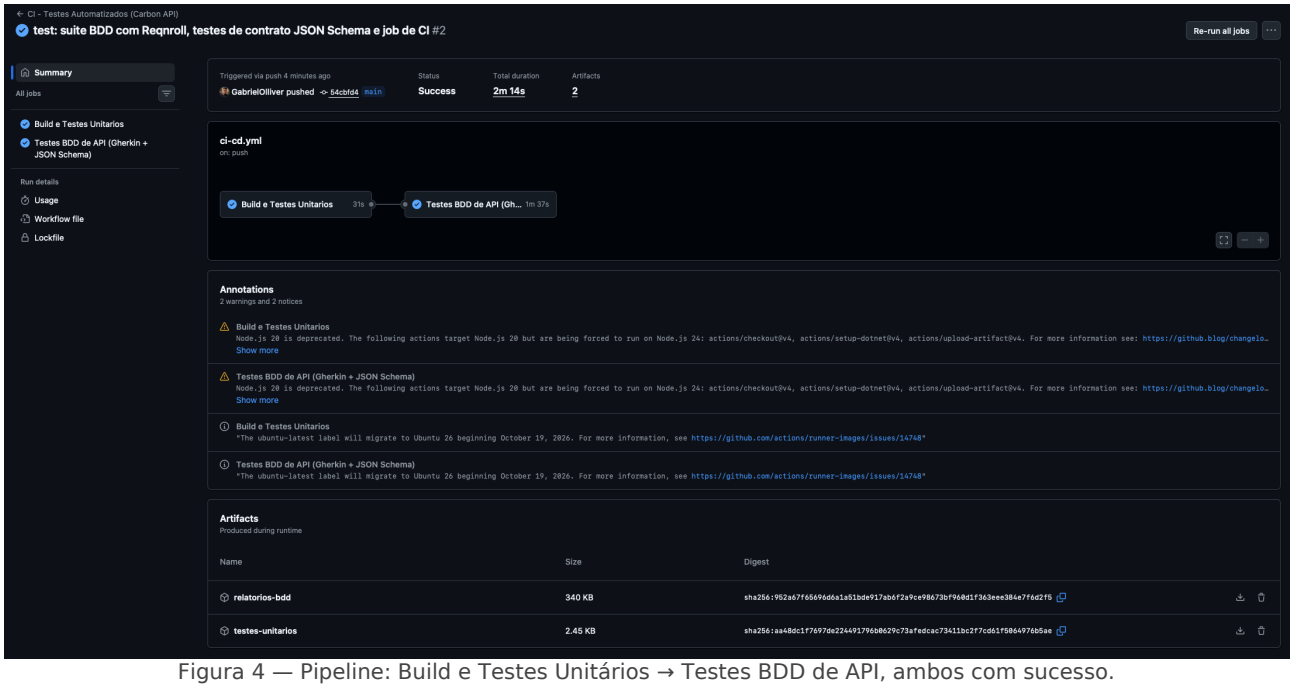


Figura 4 — Pipeline: Build e Testes Unitários → Testes BDD de API, ambos com sucesso.

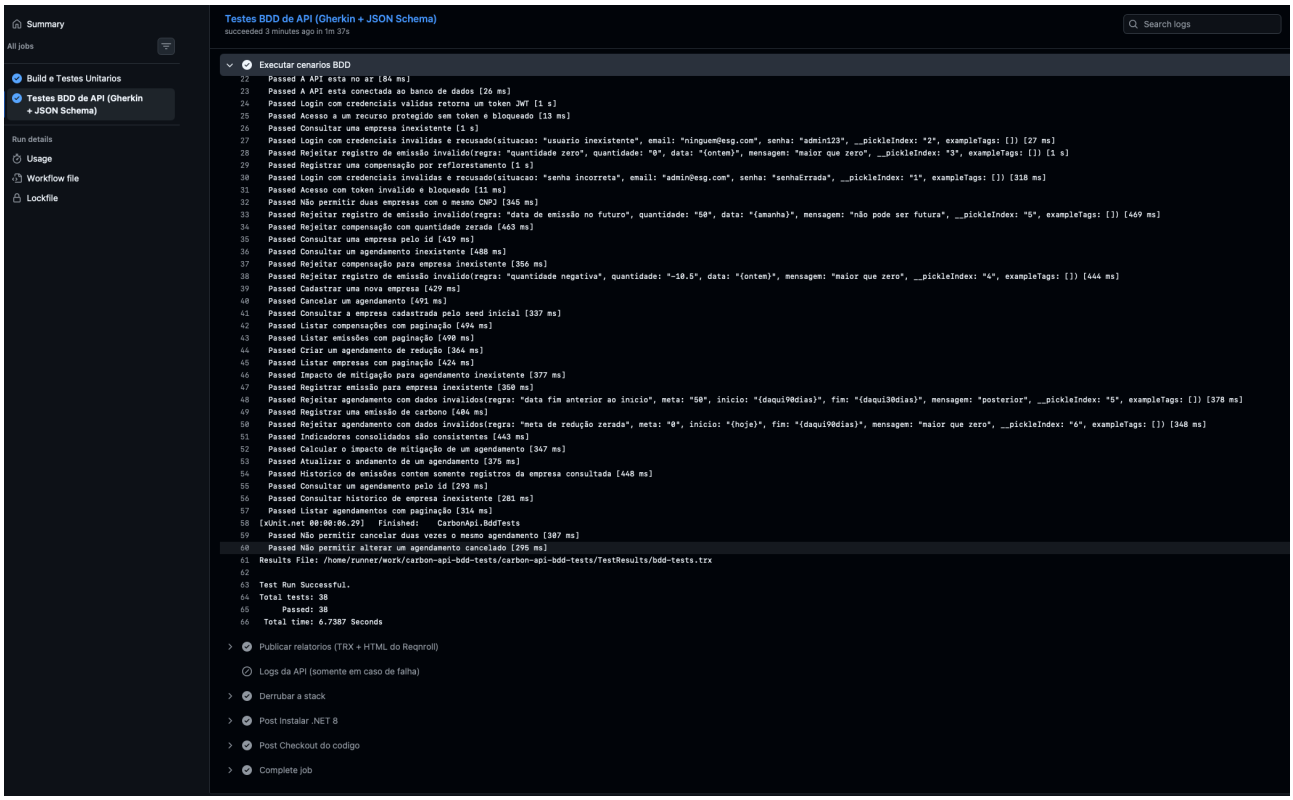


Figura 5 — Log do job BDD no runner: cenários aprovados, total 38/38.



Figura 6 — Relatórios publicados como artifacts (relatorios-bdd e testes-unitarios).

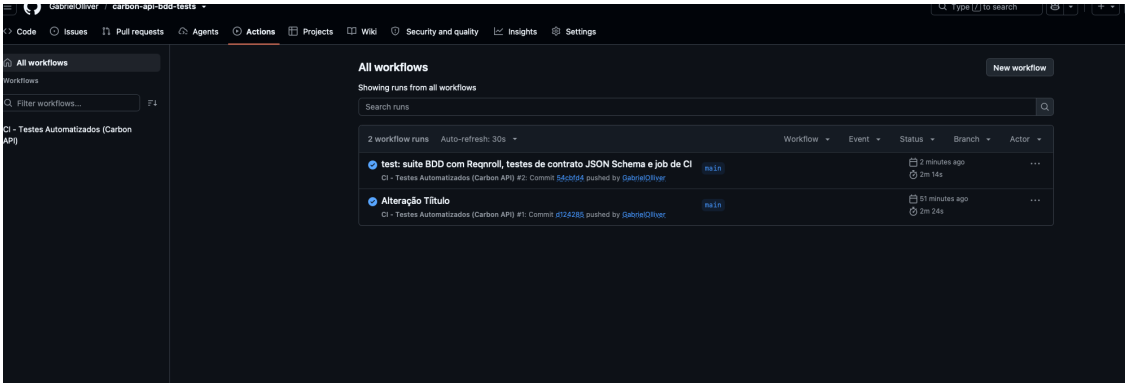


Figura 7 — Histórico de execuções do workflow no repositório.

7. Desafios e soluções

Desafio	Solução
SpecFlow, sugerido no enunciado, foi descontinuado	Adoção do Reqnroll, seu sucessor open-source compatível (mesma sintaxe Gherkin e mesmos conceitos).
Definir o resultado esperado de cada caso negativo	Leitura do ExceptionMiddleware e dos services: regra de negócio = 422, não encontrado = 404, duplicidade = 409, sem token = 401. Login inválido retorna 422 (BusinessException), e os cenários refletem o comportamento real.
Testes quebrando ao reexecutar (CNPJ duplicado)	Geração de CNPJ aleatório por cenário ({cnpjUnico}).
Dependência entre requisições (PUT/DELETE exigem um id)	ApiContext por cenário guarda {empresaId} e {agendamentoId} criados nos passos Dado.
Oracle devolve decimais com escala fixa (150.5 → 150.5000)	Comparação numérica quando o campo JSON é número, em vez de comparação textual.
Banco precisa estar pronto antes dos testes	Hook [BeforeTestRun] aguarda /health/ready; no CI um passo dedicado espera a stack antes de executar.
Segredos no CI	O pipeline gera um .env efêmero com senhas aleatórias a cada execução; nada sensível fica no repositório.

8. Como executar

```
git clone https://github.com/GabrielOlliver/carbon-api-bdd-tests.git
cd carbon-api-bdd-tests
cp .env.example .env              # ajustar senhas e JWT_SECRET
./scripts/run-bdd-tests.sh        # sobe API + Oracle, aguarda e executa os cenários

# por tag
dotnet test tests/CarbonApi.BddTests --filter "Category=governanca"
# contra outro ambiente
API_BASE_URL=http://servidor:8081 dotnet test tests/CarbonApi.BddTests
```